



第 9 章

Search Structures

搜索

Ajou University

内容

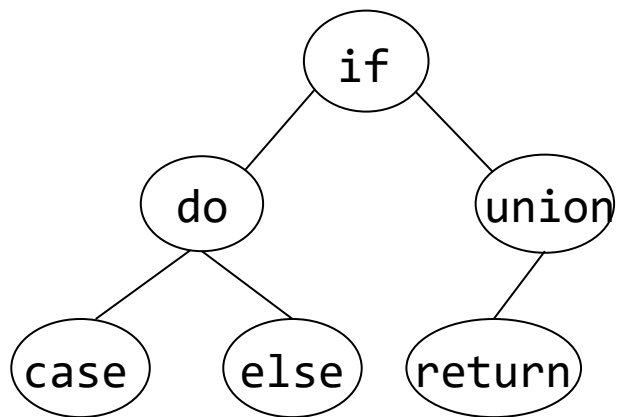
- AVL Trees (平衡二叉树)
- Red-Black Trees (红黑树)

二叉搜索树的性能

❖ 二叉搜索树的时间复杂度

- 平均情况: $O(\log n)$
- 最坏情况: $O(n)$

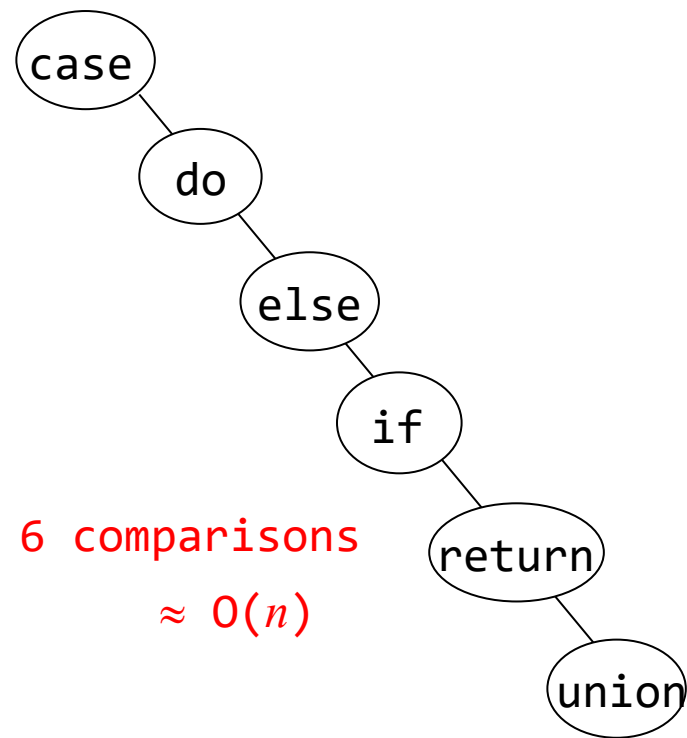
Keys={case, do, else, if, union, return}, $n=6$



3 comparisons $\approx O(\log_2 n)$

Search("while")

搜索"while", 上述结构只需要3次比较即可,
时间复杂度为 $O(\log_2 n)$



6 comparisons
 $\approx O(n)$

搜索"while", 上述结构需要6次比较,
时间复杂度为 $O(n)$, 属于最坏的情况

二叉搜索树的性能

❖ 如果我们将二叉搜索树转换为完全二叉树

- 最小化平均和最大搜索时间: $O(\log n)$
- 添加新元素所需时间显著增加, 因需重建树结构

❖ 解决方法: 构建平衡二叉树

- AVL trees (平衡二叉树)
- Red-Black trees (红黑树)

AVL Trees (平衡二叉树)

平衡二叉树(1/5)

❖ AVL树是一种平衡二叉搜索树，由Adelson-Velskii和Landis于1962年提出，以作者名字命名；

❖ 高度平衡二叉树

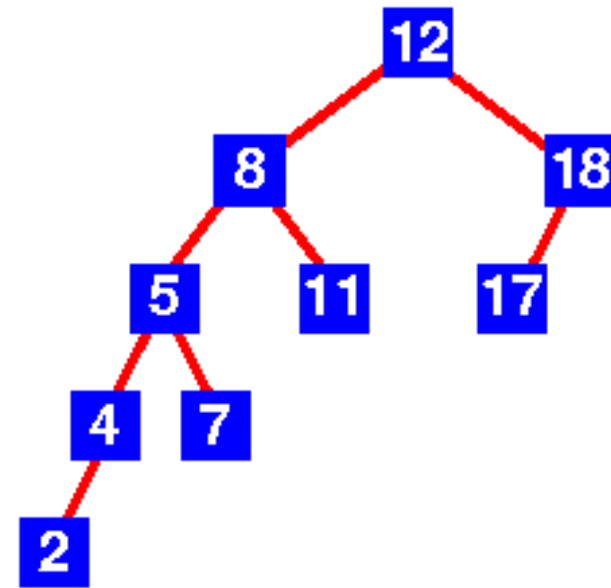
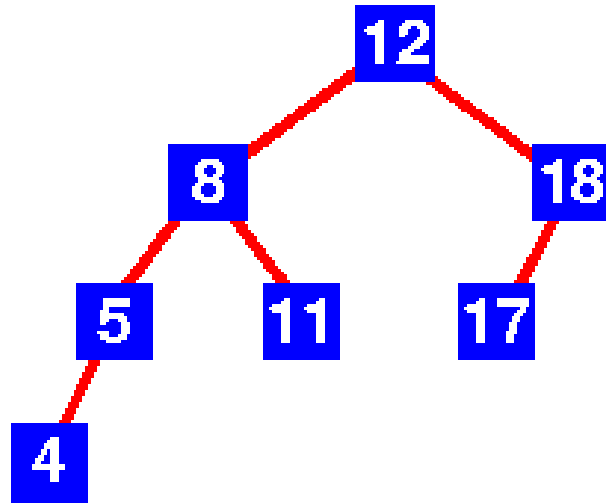
- 空二叉树为高度
- 如果T是一个非空二叉树，拥有左子树 T_L 和右子树 T_R ，T 是高度平衡的，那么
 - T_L 与 T_R 均高度平衡, 且
 - $|h_L - h_R| \leq 1$ (其中, h_L 为 T_L 的高度, h_R 为 T_R 的高度)

❖ 二叉树中节点 T 的平衡因子 BF(T)

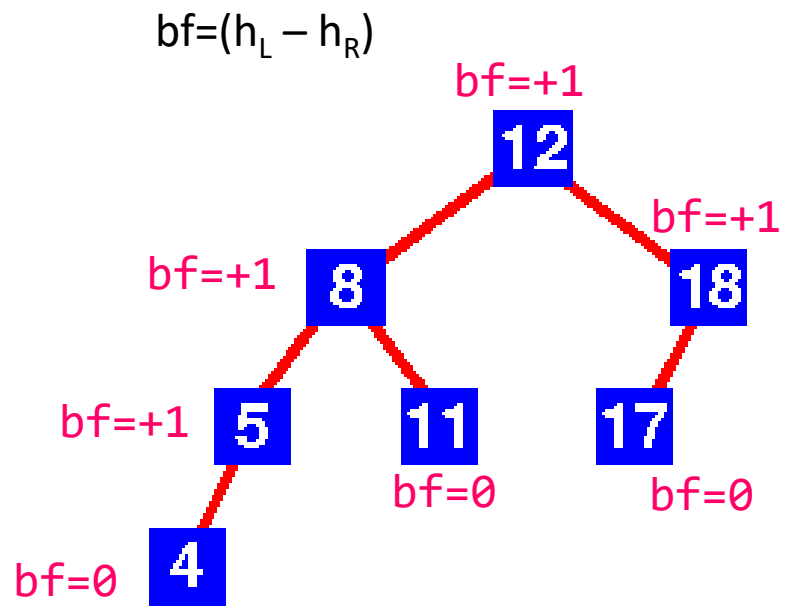
- 平衡因子为节点T的左子树高度 h_L 减去右子树 h_R 的高度，即： $(h_L - h_R)$
- 对于AVL树中的任意节点T, $BF(T) = -1, 0, \text{ or } 1$

平衡二叉树(2/5)

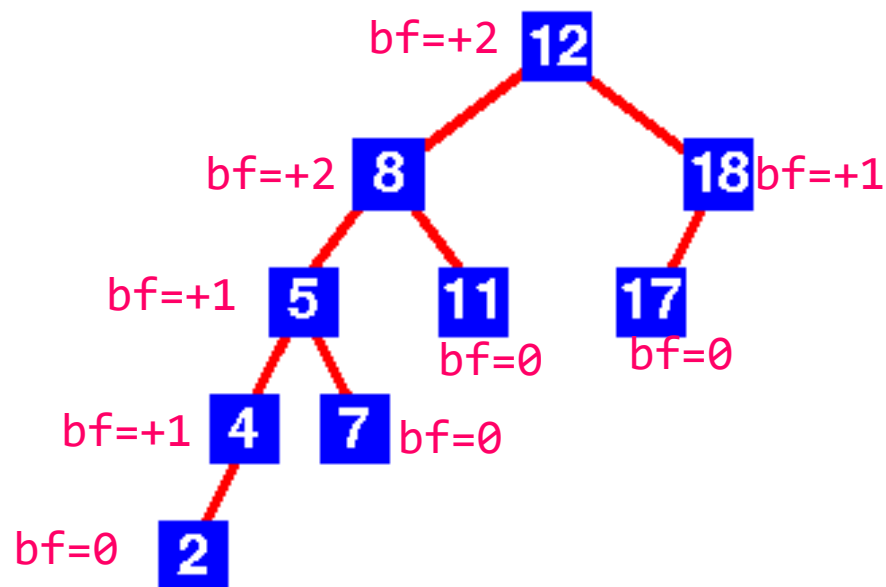
$$bf=(h_L - h_R)$$



平衡二叉树(3/5)



是AVL树(节点BF值在-1,0,1中)



不是AVL树

平衡二叉树(4/5)

❖ 旋转是搜索树中的局部操作，保持中序遍历键顺序不变。

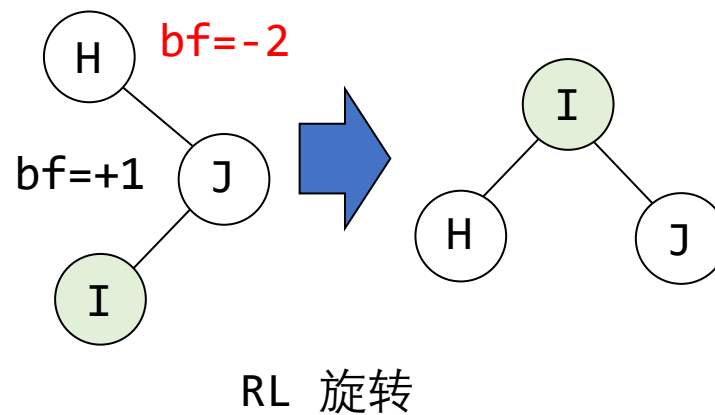
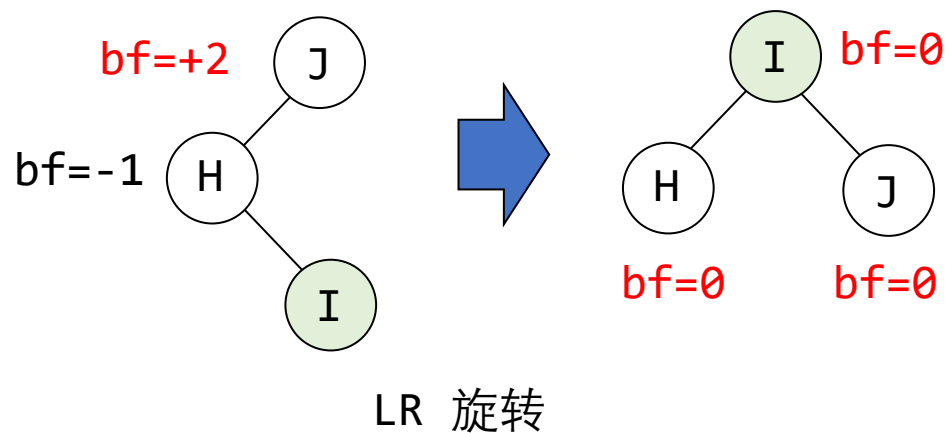
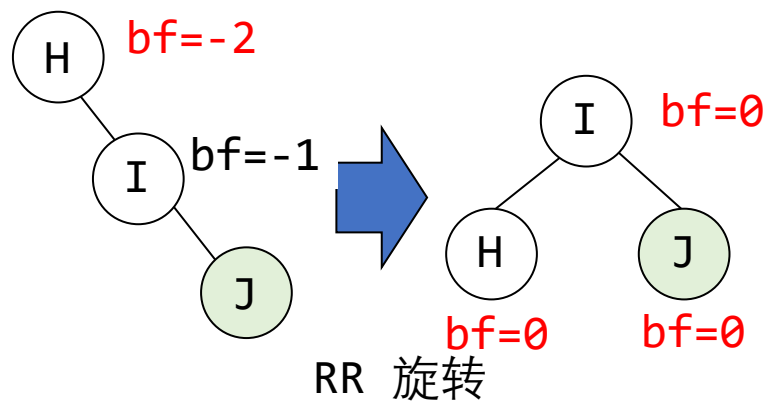
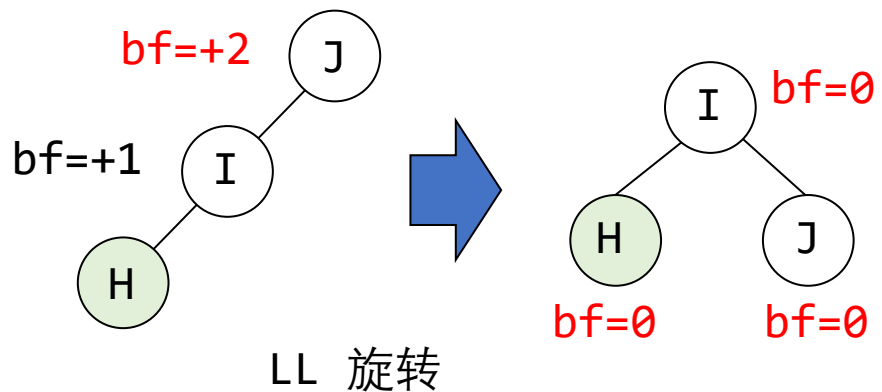
❖ 利用四种旋转重新平衡

- Y: 新插入的节点,
- A: Y的最近祖先, 其平衡因子变为 ± 2
- LL : Y插入A左子树的左子树
- LR : Y插入A左子树的右子树
- RR : Y插入A右子树的右子树
- RL : Y插入A右子树的左子树

❖ AVL树中插入节点

- 时间复杂度: $O(h)$, 其中, h 是插入前树的高度。

平衡二叉树(5/5)



平衡二叉树插入元素案例 (1/8)

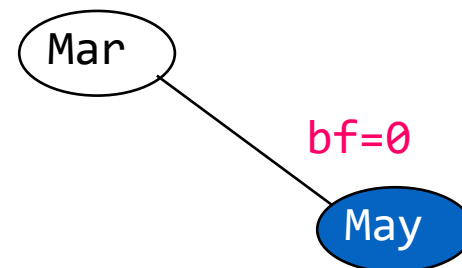
(1) insert March(三月)

bf=0

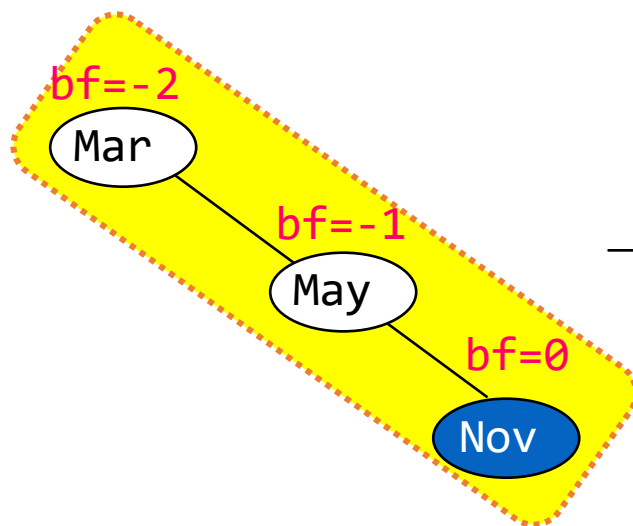


(2) insert May(五月)

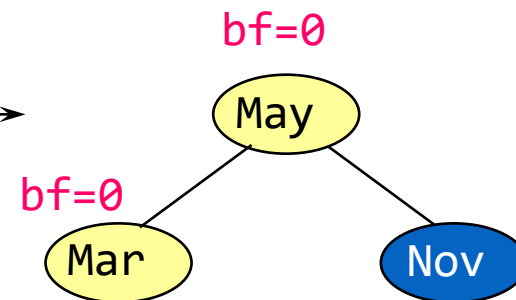
bf=-1



(3) insert November(十一月)

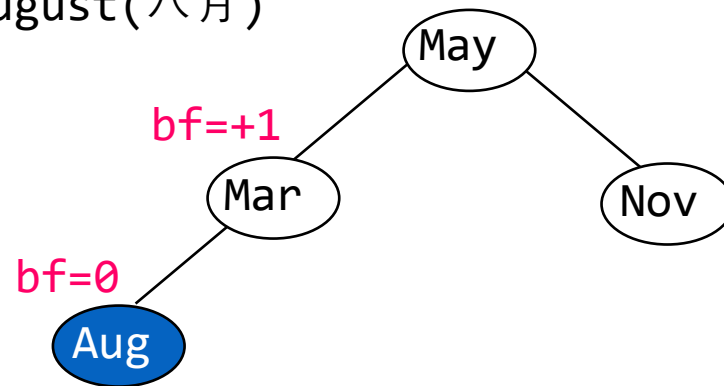


RR 旋转

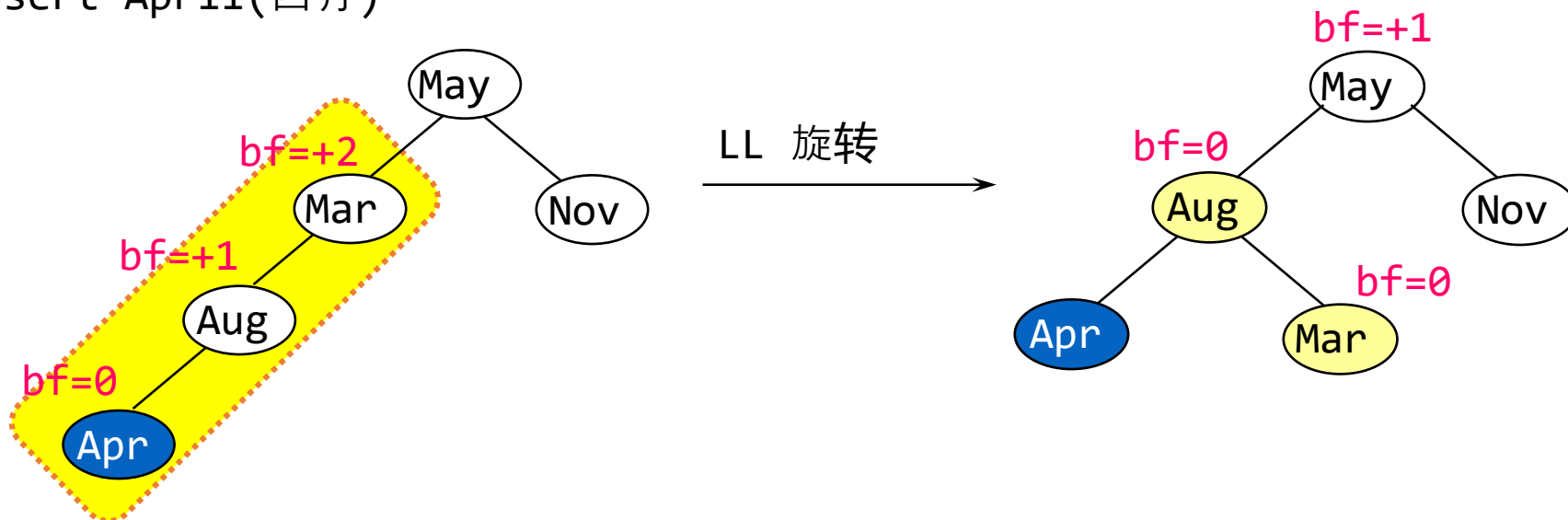


平衡二叉树插入元素案例 (2/8)

(4) insert August(八月)

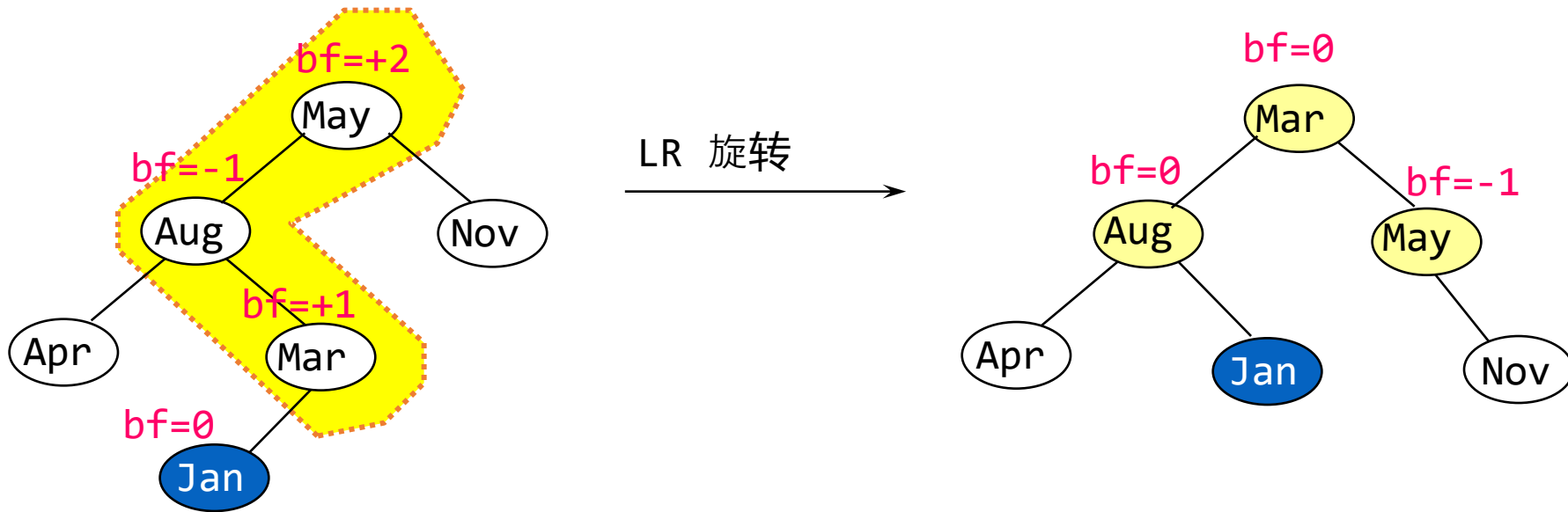


(5) insert April(四月)



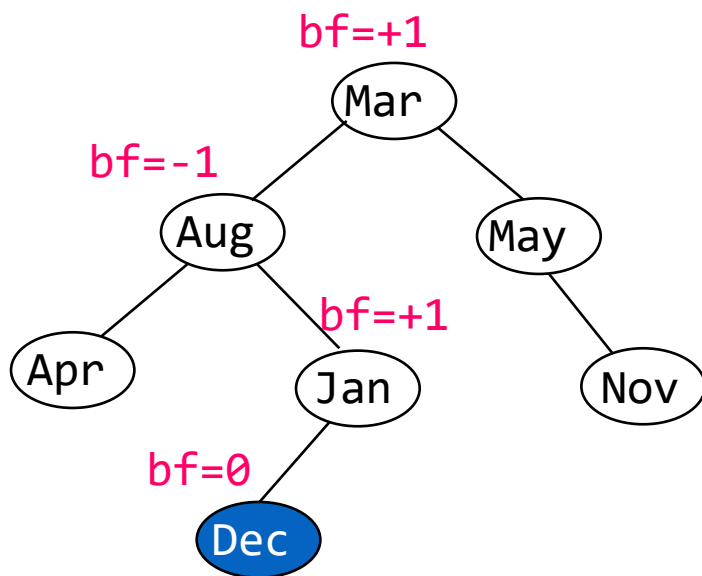
平衡二叉树插入元素案例 (3/8)

(6) insert January(一月)

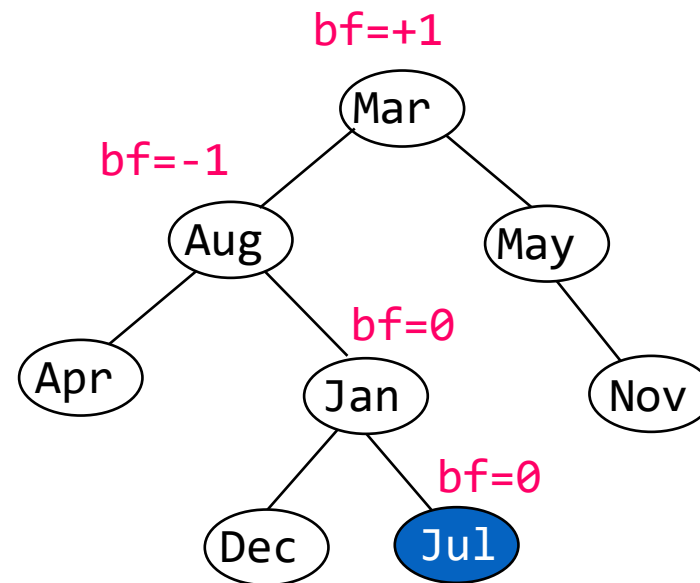


平衡二叉树插入元素案例 (4/8)

(7) insert December(十二月)

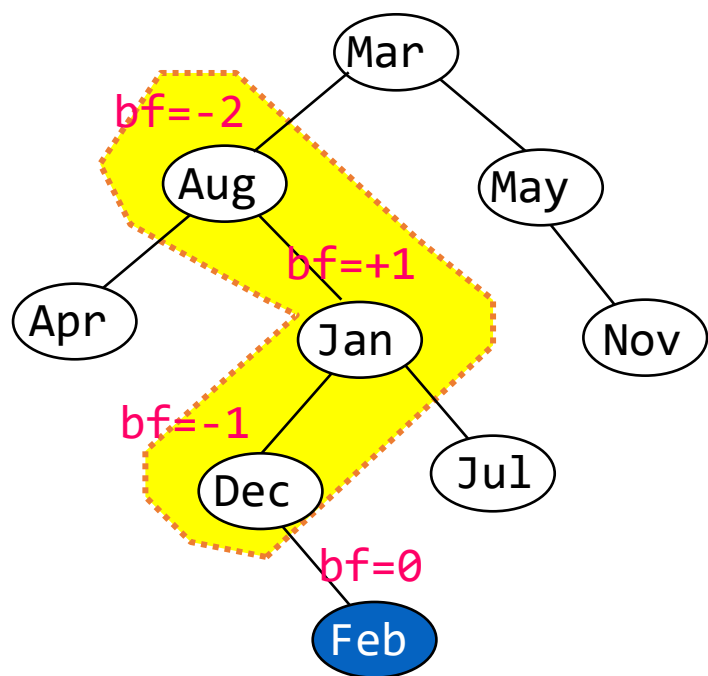


(8) insert July(七月)

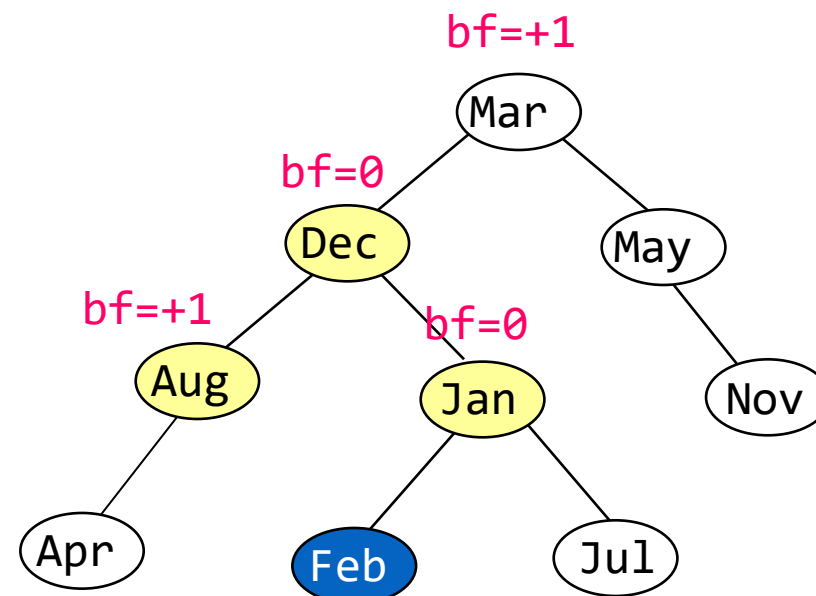


平衡二叉树插入元素案例 (5/8)

(9) insert February(二月)

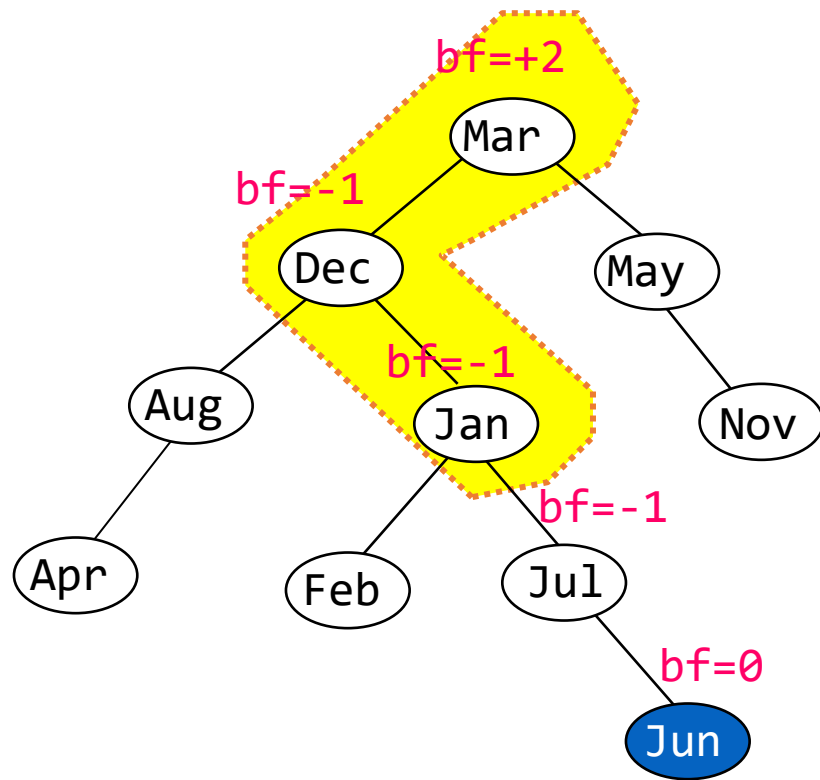


RL 旋转

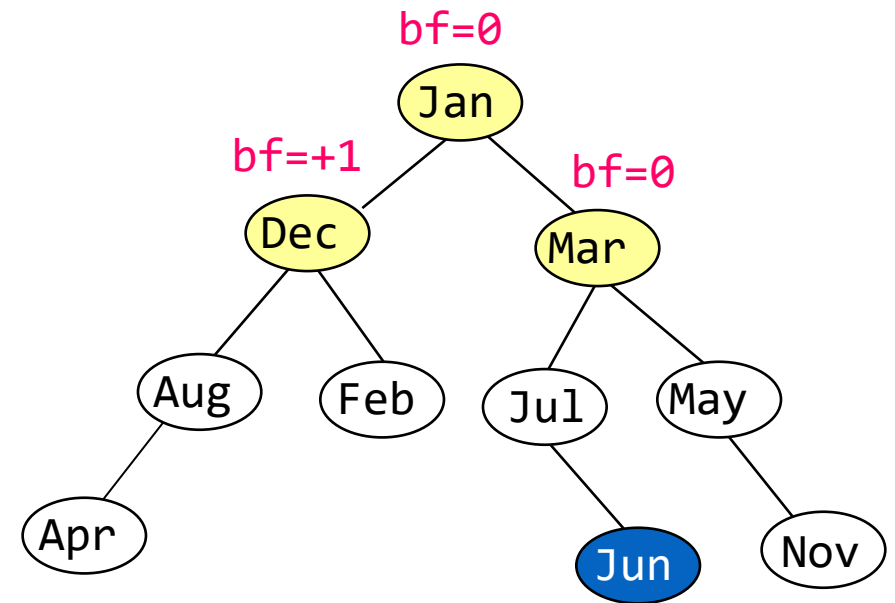


平衡二叉树插入元素案例 (6/8)

(10) insert June(六月)

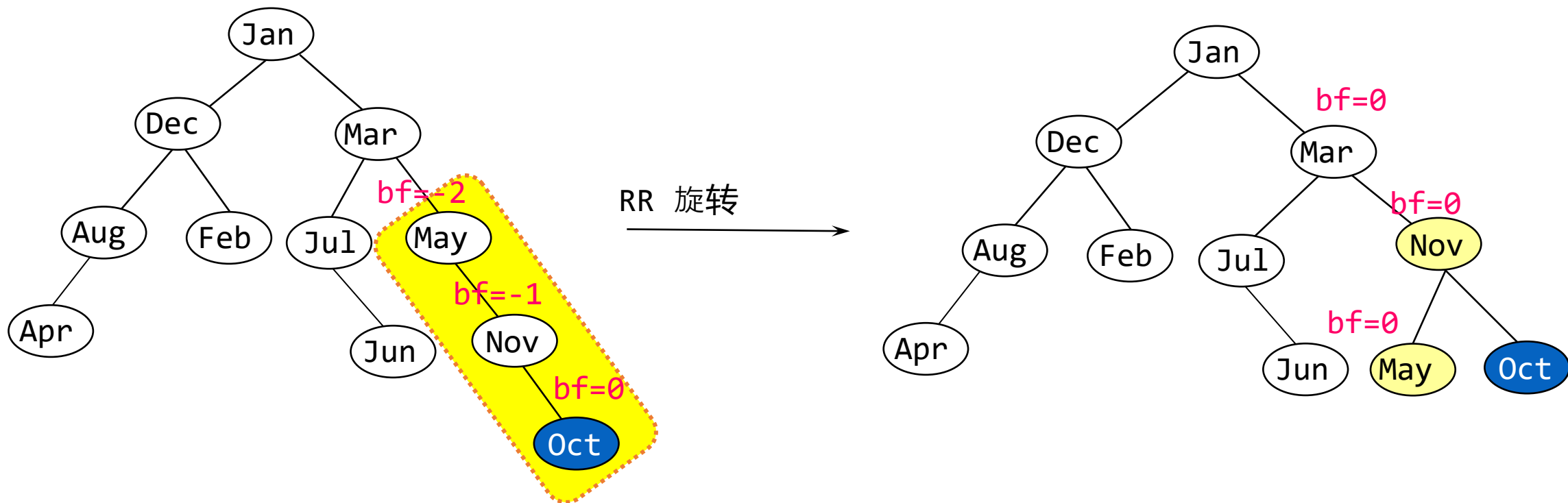


LR 旋转



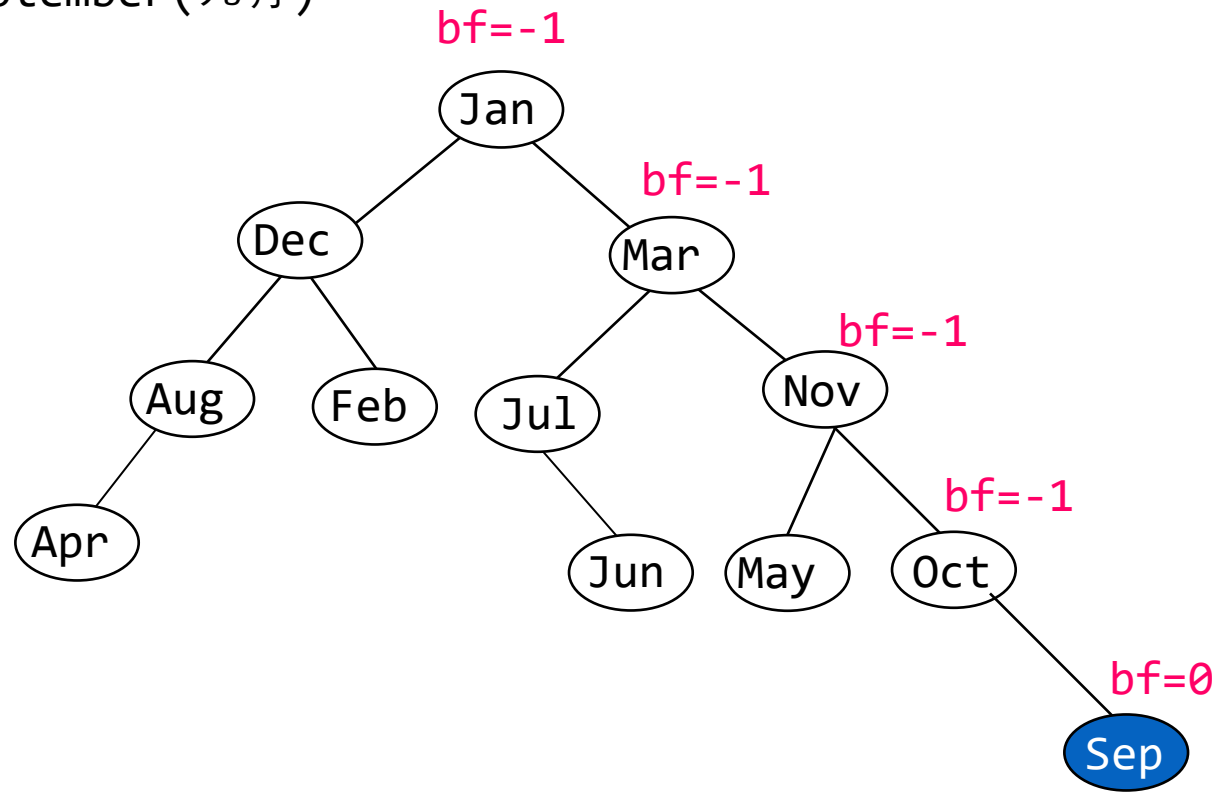
平衡二叉树插入元素案例 (7/8)

(11) insert October(十月)



平衡二叉树插入元素案例 (8/8)

(12) insert September(九月)



平衡二叉树C语言实现 (1/4)

```
#define struct {  
    int key;  
} element;  
  
typedef struct treeNode *treePointer;  
struct treeNode {  
    treePointer leftChild;  
    element data;  
    short int bf;  
    treePointer rightChild;  
};  
  
int unbalanced = FALSE;  
treePointer root=NULL;
```

平衡二叉树C语言实现 (2/4)

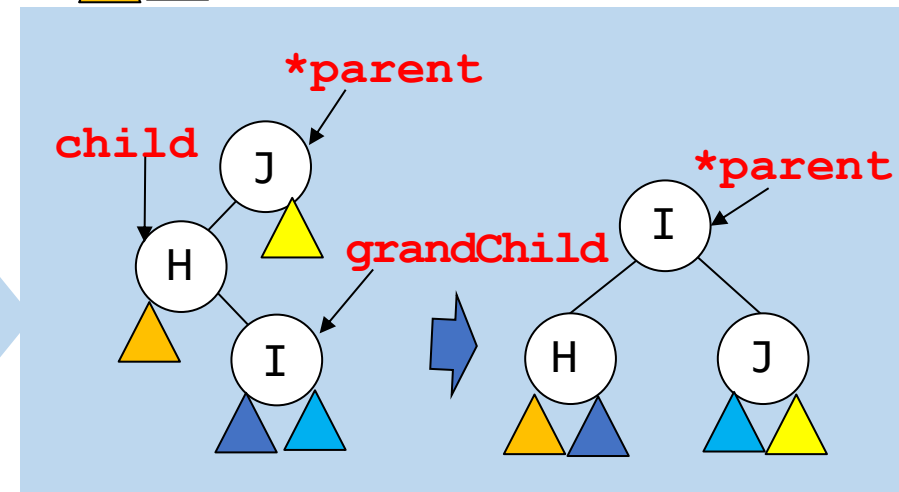
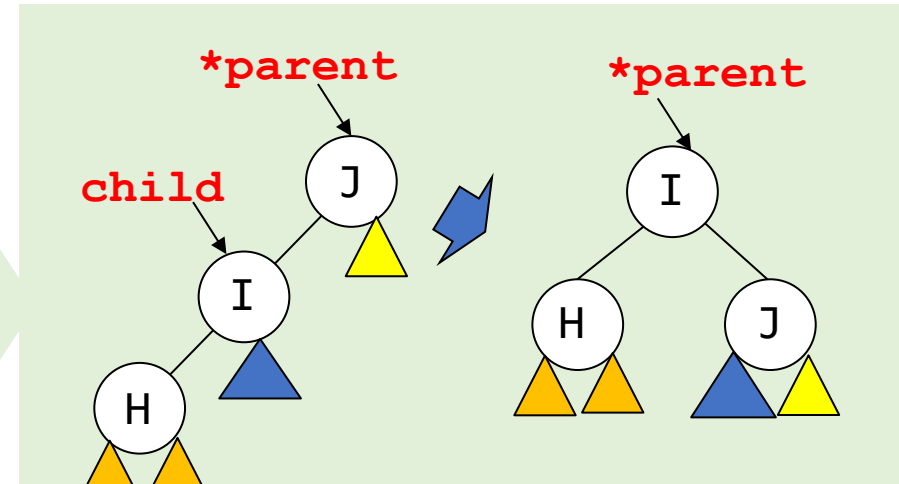
```
void avlInsert(treePointer *parent, element x, int *unbalanced) {
    if (!*parent) { /* insert element into null tree */
        *unbalanced = TRUE;
        *parent = (treePointer) malloc(sizeof(treeNode));
        (*parent)->leftChild = (*parent)->rightChild = NULL;
        (*parent)->bf = 0;
        (*parent)->data = x;
    } else if (x.key < (*parent)->data.key) {
        avlInsert(&(*parent)->leftChild, x, unbalanced);
        if (*unbalanced) /* left branch has grown higher */
            switch ((*parent)->bf) {
                case -1: (*parent)->bf = 0;
                    *unbalanced = FALSE; break;
                case 0: (*parent)->bf = 1; break;
                case 1: leftRotation(parent, unbalanced);
            }
    }
}
```

平衡二叉树C语言实现 (3/4)

```
    else if(x.key > (*parent)->data.key) {
        avlInsert(&(*parent)->rightChild, x, unbalanced);
        if(*unbalanced) /*right branch has grown higher */
            switch((*parent)->bf) {
                case 1: (*parent)->bf = 0;
                        *unbalanced = FALSE; break;
                case 0: (*parent)->bf = -1; break;
                case -1: rightRotation(parent, unbalanced);
            }
    } else { /* x.key == (*parent)->data.key */
        *unbalanced = FALSE;
        printf ("The key is already in the tree\n");
    }
} /* end of avlInsert */
```

平衡二叉树C语言实现 (4/4)

```
void leftRotation(treePointer* parent, int* unbalanced)
{
    treePointer grandChild, child;
    child = (*parent) -> leftChild;
    if (child->bf == 1) { /*LL rotation */
        (*parent) -> leftChild = child->rightChild;
        child->rightChild = *parent;
        (*parent)->bf = 0;
        (*parent) = child;
    } else { /* LR rotation */
        grandChild=child->rightChild;
        child->rightChild = grandChild->leftChild;
        grandChild->leftChild=child;
        (*parent)->leftChild = grandChild->rightChild;
        grandChild->rightChild= *parent;
        switch (grandChild->bf) {
            case 1: (*parent)->bf=-1; child->bf=0; break;
            case 0: (*parent)->bf=child->bf=0; break;
            case -1: (*parent)->bf=0; child->bf=1;
        }
        *parent=grandChild;
    }
    (*parent)->bf=0; *unbalanced=FALSE;
}
```

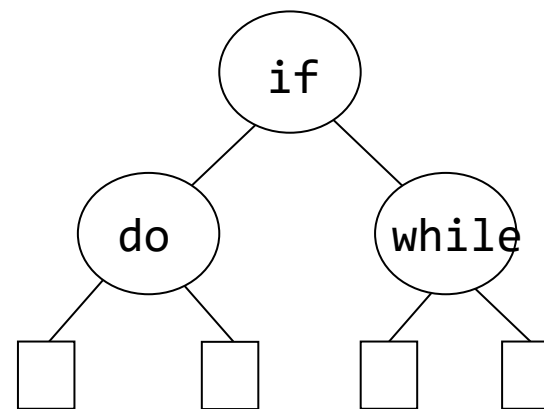


Red-Black Trees

红黑树

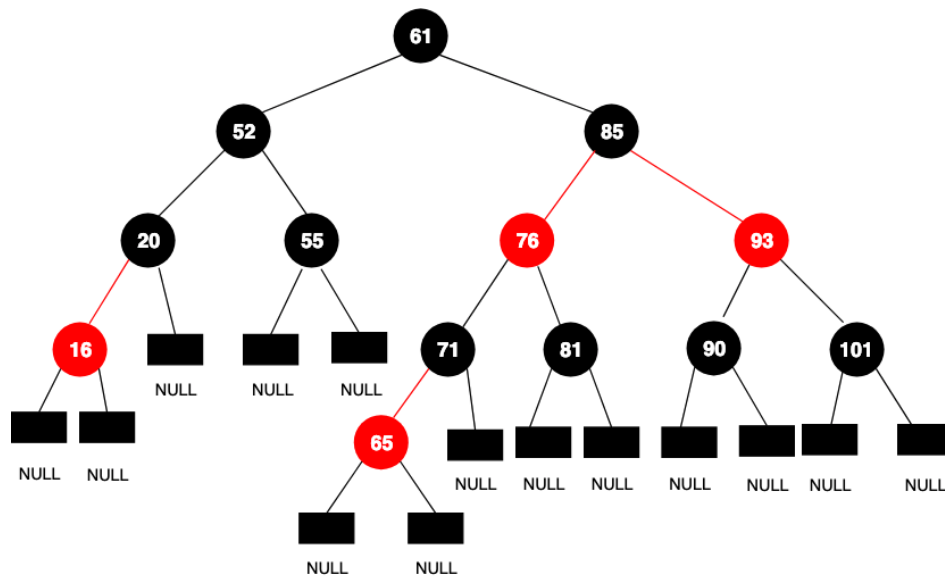
扩展二叉搜索树

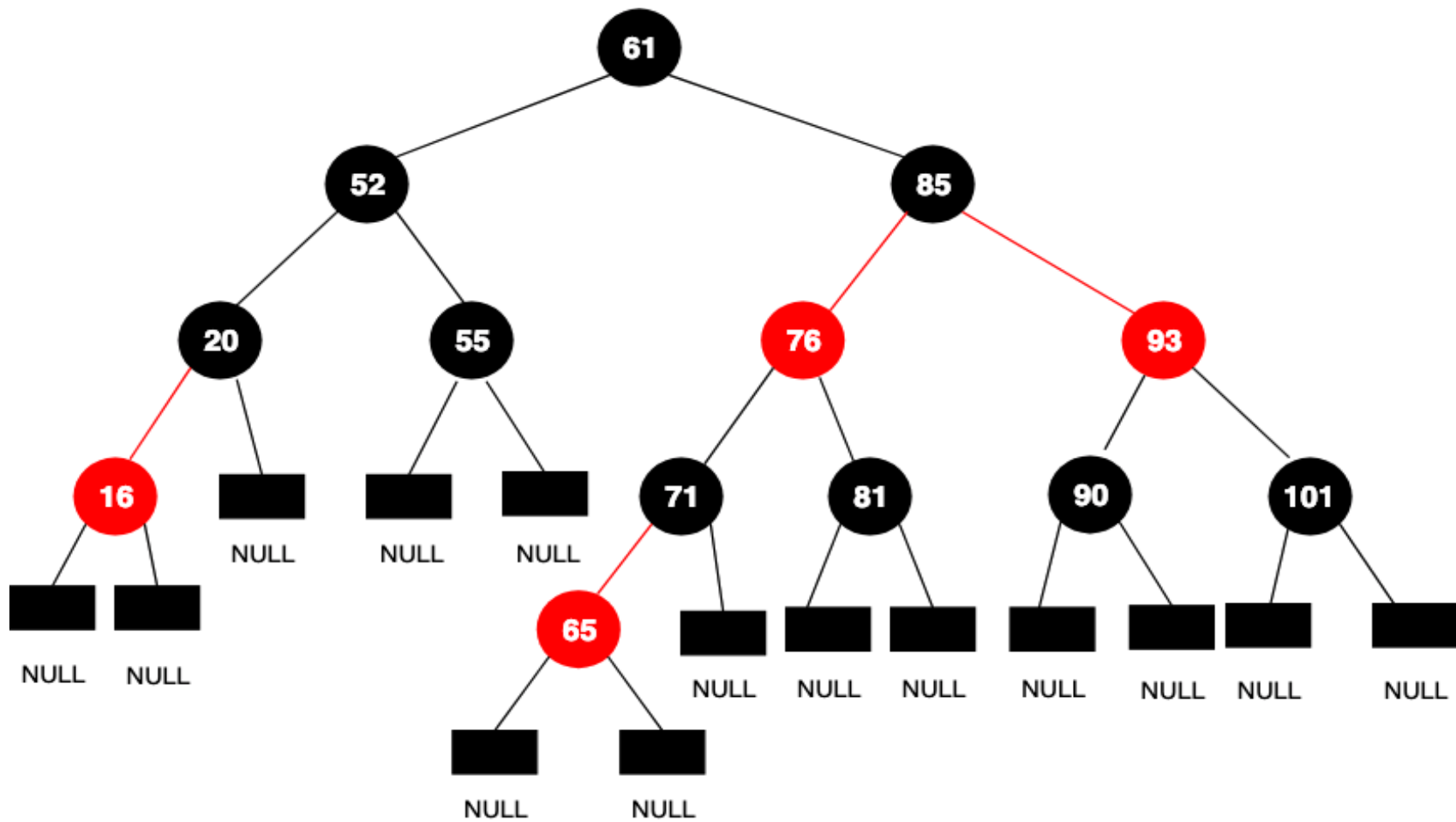
- 在每个空链接处放置特殊方形节点
 - 外部节点：不属于原始树
 - 内部节点：属于原始树



红黑树

- 红黑树是一种二叉搜索树，其中每个节点都被染成红色或黑色。
- 红黑树是维护二叉搜索树的一种首选方法





❖ 属性

RB1:根节点和所有外部节点均被染成黑色。

RB2:从根节点到外部节点的任何路径上不存在连续的两个红色节点。

RB3:从根节点到外部节点的每条路径上黑色节点的数目相同。

红黑树

- ❖ 具有 n 个内部节点的红黑树的高度最多为 $2 \cdot \log_2(n+1)$
- ❖ 近似平衡支持插入和删除操作，时间复杂度为
→ $O(\log n)$

红黑树C语言实现

```
typedef enum {red, black} color;
typedef struct redBlack *redBlackPtr;
typedef struct redBlack {
    element data;
    redBlackPtr leftChild;
    redBlackPtr rightChild;
    color Color;
}
```

红黑树插入 (1/2)

- 在 BST(二叉搜索树)中按常规方式插入新节点a

```
if the tree is empty:  
    a->color = black (see RB1)  
else:  
    a->color = red
```

RB1: 根节点和所有外部节点均被染成黑色

RB2: 从根节点到外部节点的任何路径上不存在连续的两个红色节点

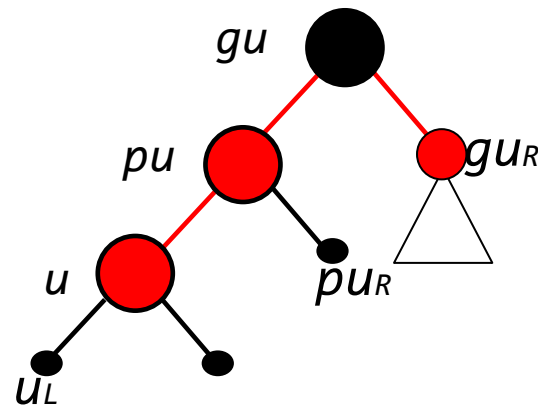
RB3: 从根节点到外部节点的每条路径上黑色节点的数目相同

将新节点设为黑色必然违反RB3规则，而将其设为红色可能不会违反RB2规则。

- 如果将新节点设为红色导致RB2规则被违反
->树将变得不平衡

红黑树插入 (2/2)

- 类型为XYc的不平衡情况 (X和Y可为L或R, c可为r (红色) 或b (黑色))
- LLb, LLr, LRb, LRr, RRb, RRr, RLb, RLr



u: 新节点
pu: u的父节点
gu: u的祖父节点

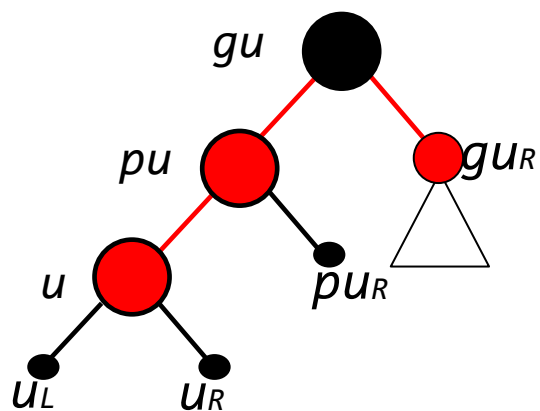
LLr : pu 是 gu 的左子节点, u 是 pu 的左子节点, 且叔节点是红色

红黑树插入 (2/2)

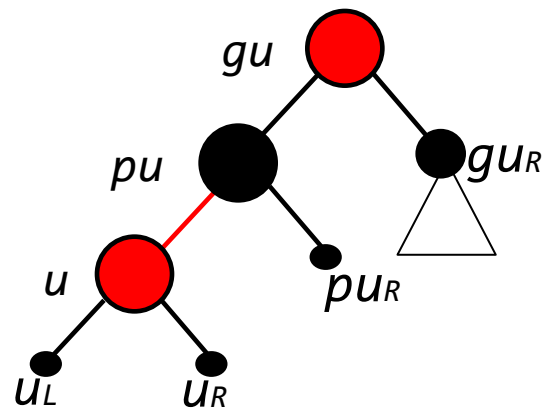
- LLb, LLr, LRb, LRr, RRb, RRr, RLb, RLr
 - LLb : pu 是 gu 的左子节点, u 是 pu 的左子节点, 且叔节点为黑色
 - LLr : pu 是 gu 的左子节点, u 是 pu 的左子节点, 且叔节点为红色
 - LRb : pu 是 gu 的左子节点, u 是 pu 的右子节点, 且叔节点为黑色
 - LRr : pu 是 gu 的左子节点, u 是 pu 的右子节点, 且叔节点为红色
- RRb, RRr, RLb, RLr
- 类型为XYr的不平衡通过改变颜色处理
- 类型为XYb的不平衡需要进行旋转

XYr不平衡 - 重新着色

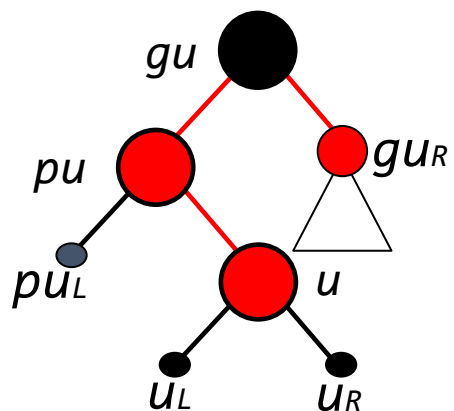
RB1: 根节点和所有外部节点均被染成黑色
RB2: 从根节点到外部节点的任何路径上不存在连续的两个红色节点
RB3: 从根节点到外部节点的每条路径上黑色节点的数目相同



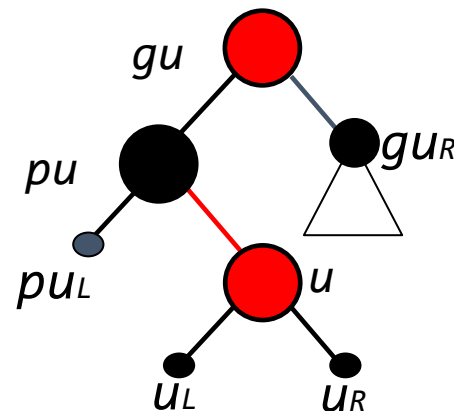
(a) LLr 不平衡



(b) LLr着色变化后



(c) LRr不平衡



(d) LRr着色变化后

XYr不平衡 - 重新着色

将新节点的父节点和叔节点颜色改为黑色，祖父节点颜色改为红色。

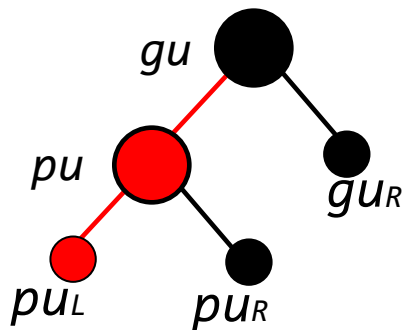
- 若祖父节点为根节点，则将其颜色改为黑色
- 若祖父节点颜色变为红色后再次违反RB2规则，则重复重新着色或调整过程，直至违反被解决

RB1:根节点和所有外部节点均被染成黑色。

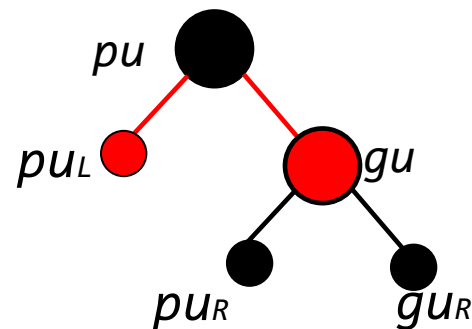
RB2:从根节点到外部节点的任何路径上不存在连续的两个红色节点。

RB3:从根节点到外部节点的每条路径上黑色节点的数目相同。

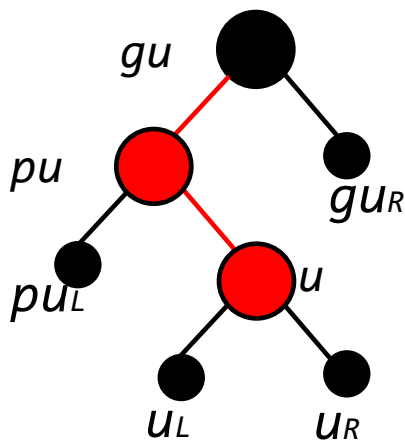
XYb不平衡 - 旋转



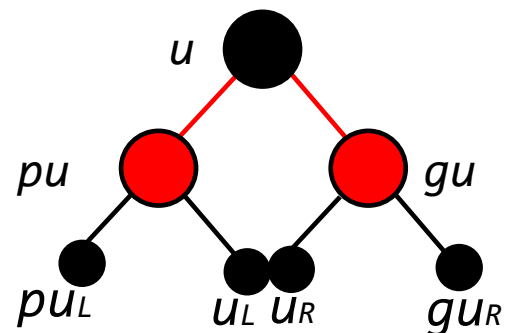
(a) LLb不平衡



(b) LLb旋转后



(c) LRb不平衡



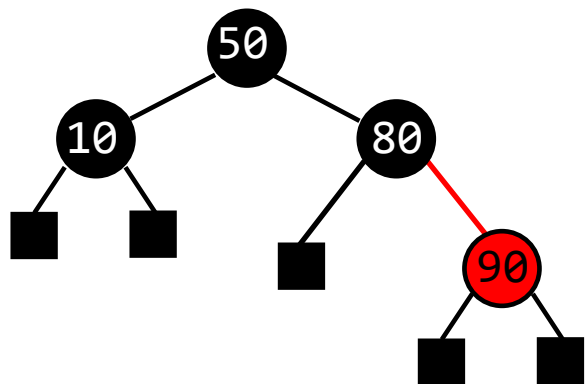
(d) LRb旋转后

XYb 不平衡 - 旋转

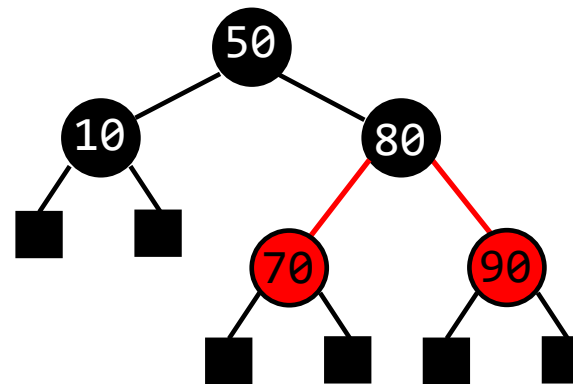
- 进行旋转（LRB 不平衡 -> LR 旋转）。
- 将新父节点的颜色改为黑色，其他节点改为红色。

红黑树插入案例 (1/3)

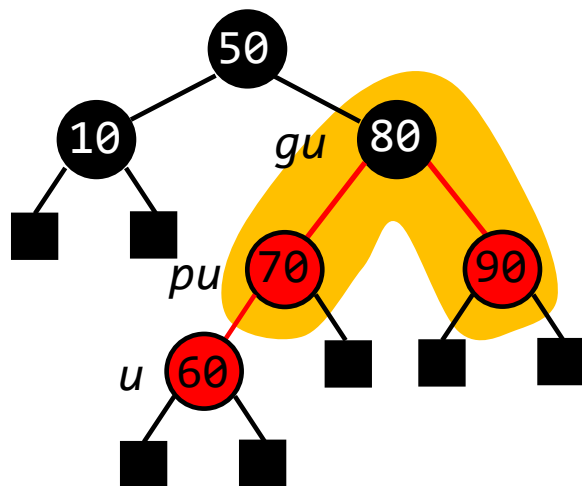
(a) 初始



(b) 插入 70

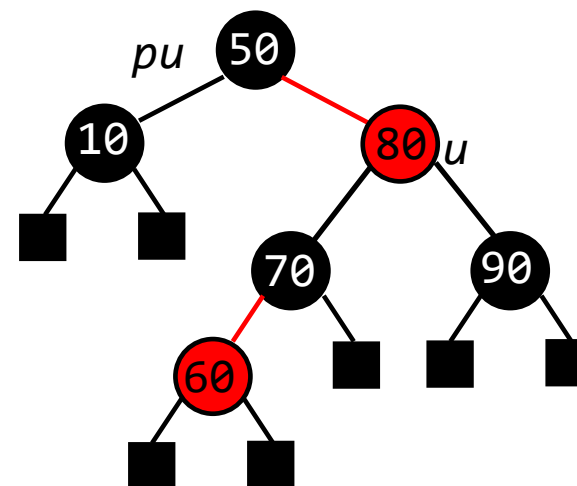


(c) 插入 60



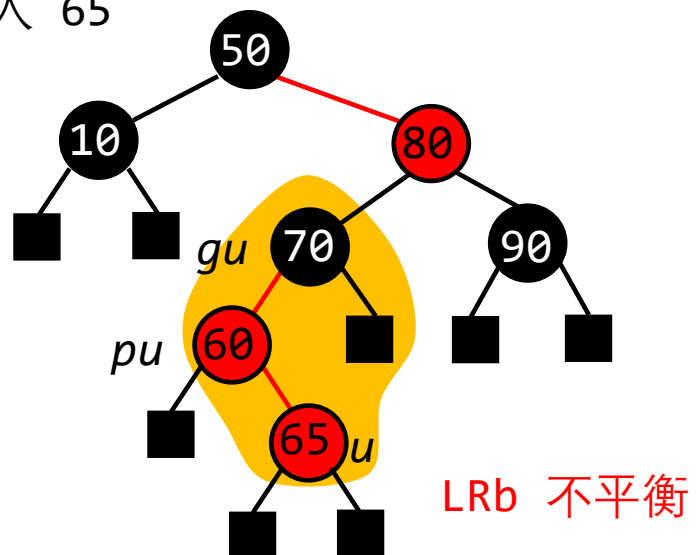
LLr 不平衡

(d) LLr颜色改变

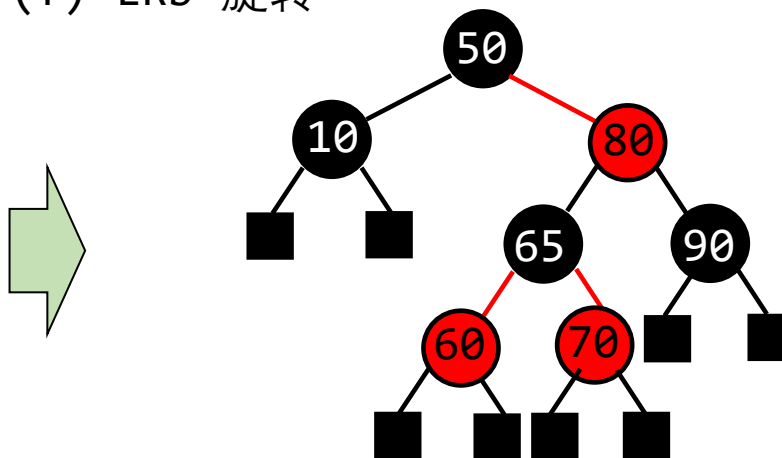


红黑树插入案例 (2/3)

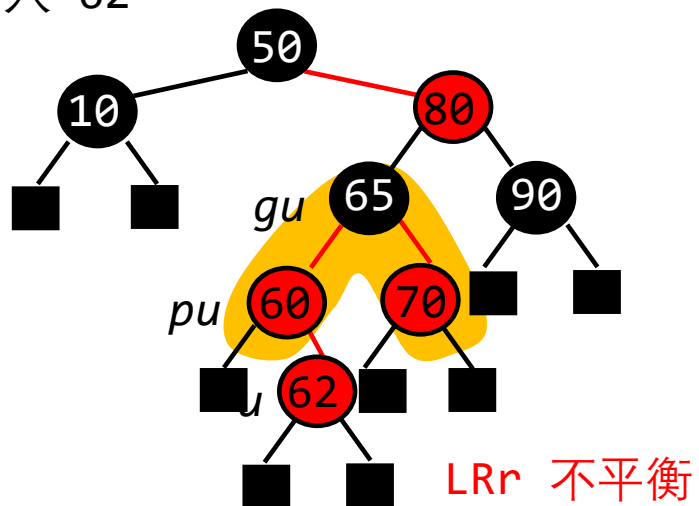
(e) 插入 65



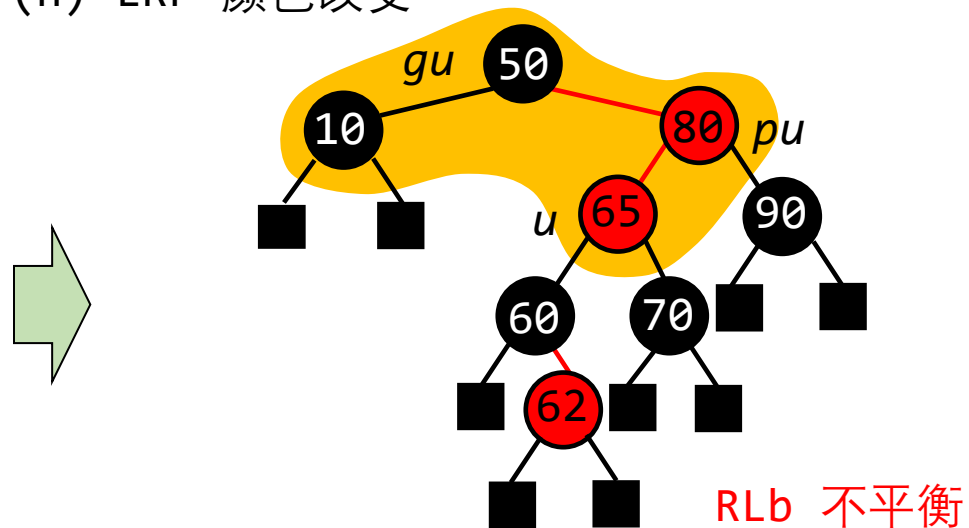
(f) LRb 旋转



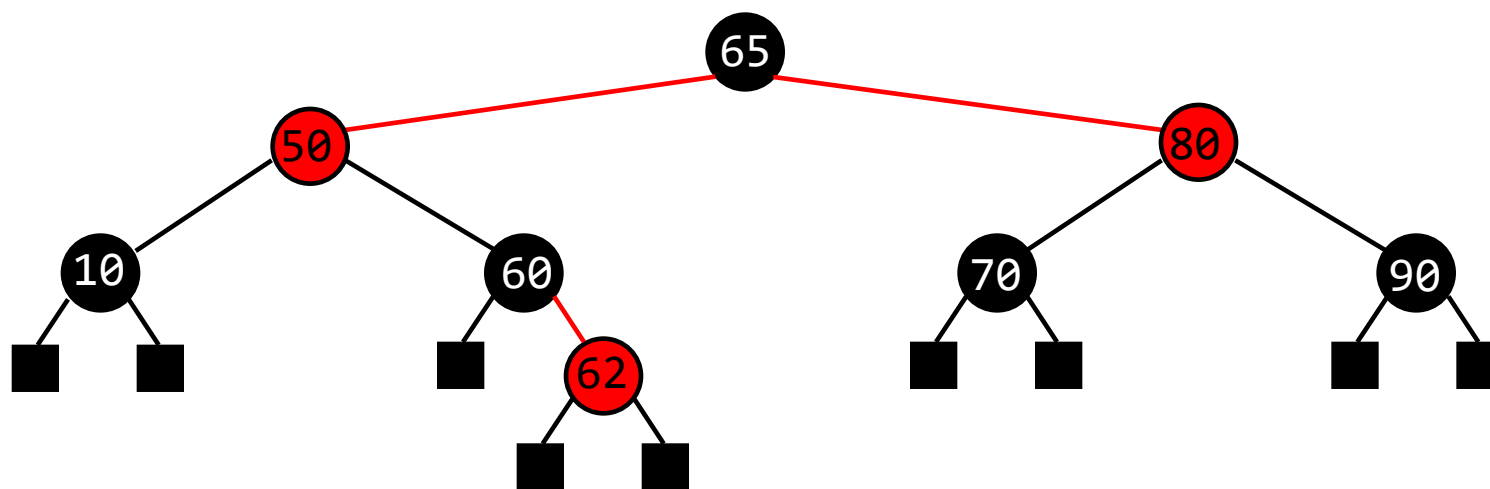
(g) 插入 62



(h) LRr 颜色改变



红黑树插入案例 (3/3)



(i) RLb 旋转

平衡二叉树 与 红黑树对比 (1/2)

- 字典创建时间（以毫秒为单位）

Number of Entries	Hash Table	AVL Tree	Red-Black Tree
5	2	30	24
10	1	4	1
20	1	2	1
40	1	2	1
80	1	3	2
160	2	6	2
320	2	18	33
640	2	62	4
1280	2	238	9
2560	4	975	14
5120	6	3704	38

NEYER, Mark P. A Comparison of Dictionary Implementations. 2009.

平衡二叉树 与 红黑树对比 (2/2)

- 字典查找时间（以毫秒为单位）

Number of Entries	Hash Table	AVL Tree	Red-Black Tree
5	0	1	1
10	0	1	1
20	0	2	2
40	1	5	4
80	1	13	9
160	3	28	19
320	6	64	41
640	10	143	92
1280	21	310	202
2560	44	679	489
5120	91	1517	1002

NEYER, Mark P. A Comparison of Dictionary Implementations. 2009.